Phase 4 – Host Forensics (Netstat)

1. Tasks

- **Command netstat -rn**

  The netstat -rn command displays the kernel's routing table, which dictates the path network packets take to reach their destination. It uses the numeric flag to show raw IP addresses and port numbers instead of resolving hostnames, ensuring the output is fast and technically precise. In a forensic context, this is used to identify the Default Gateway (listed under the 0.0.0.0 destination) and any unauthorized static routes that might be redirecting traffic. By examining the "Flags" column, you can verify if a route is active and specifically designated as a gateway.

Destination	**Gateway**	**Genmask**	**Interface**	**What it means**
0.0.0.0	10.0.2.1	0.0.0.0	eth0	All traffic destined for the internet or external networks goes here.
10.0.2.0	0.0.0.0	255.255.255.0	eth0	This is your local subnet. Traffic to other VMs (like your Windows VM) stays here.
172.17.0.0	0.0.0.0	255.255.0.0	docker0	This is the internal network for your Docker containers.

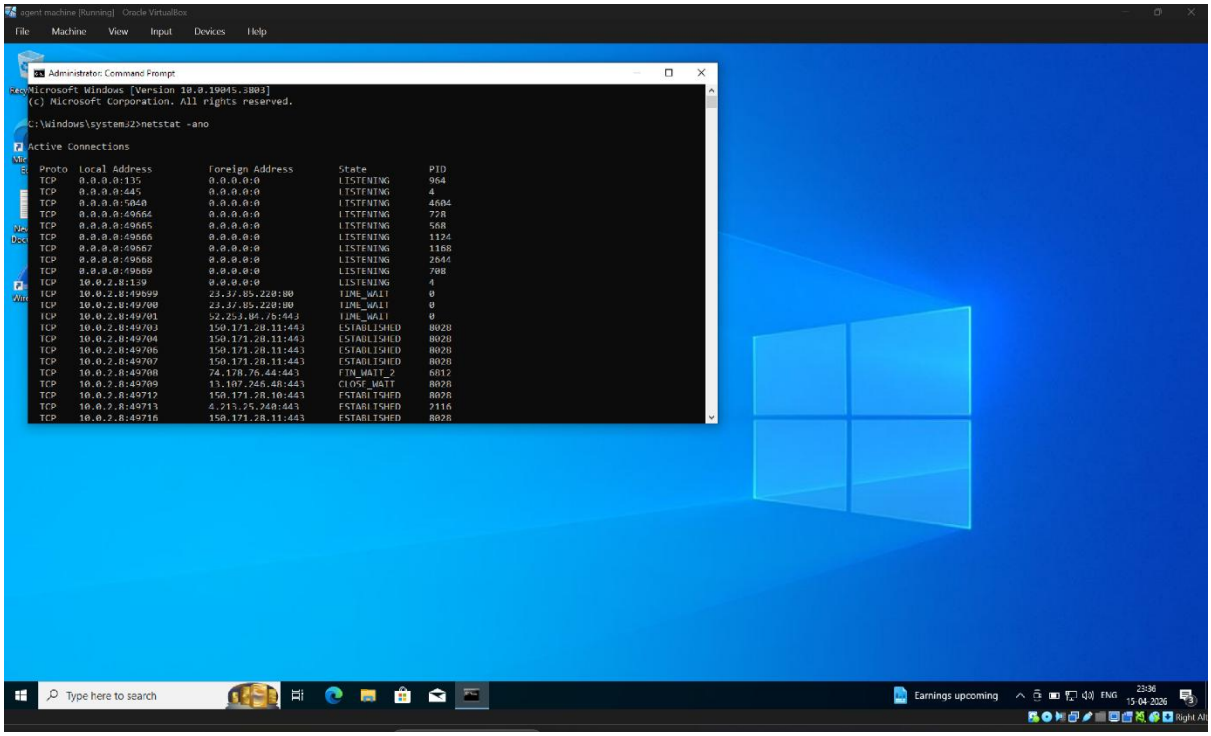
- **Command netstat -tulpn**

  The netstat -tulpn command identifies all listening services on a system by displaying active TCP (-t) and UDP (-u) ports. It specifically filters for ports in a listening state (-l), showing which programs are waiting for incoming connections. By including the program flag (-p), it links each open port to a specific Process ID (PID), which is vital for identifying "who" is responsible for network traffic. The numeric flag (-n) forces the output to show raw port numbers (like 21 or 80) instead of service names, ensuring technical accuracy and speed. In forensics, this is your primary tool for mapping the attack surface and spotting unauthorized or "rogue" services running in the background.

Port	**Protocol**	**Program Name**	**Description**
23	TCP/TCP6	inetd	**Telnet Server.** This is a critical clear-text vulnerability.
80	TCP6	apache2	**Web Server.** This is hosting your DVWA instance.
1514/1515	TCP/TCP6	docker-proxy	Likely the **Wazuh** manager running in a Docker container.
9200	TCP/TCP6	docker-proxy	Likely **Elasticsearch** (part of your SIEM stack).
8443	TCP/TCP6	docker-proxy	A common port for secure web dashboards.
514	UDP/UDP6	docker-proxy	**Syslog server** for collecting network logs.

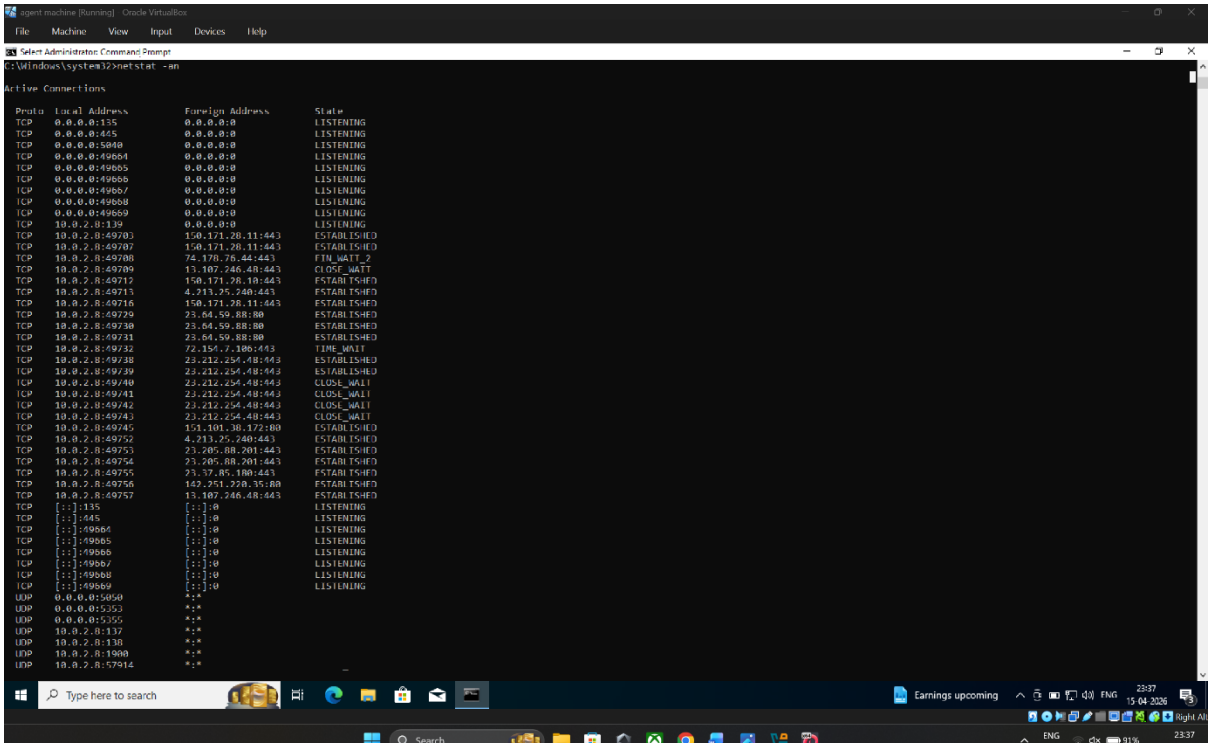
- **Command netstat -ano**

  The netstat -ano command lists all active network connections and listening ports on a Windows system. The -a flag displays all connections, -n shows them in numerical format for speed, and -o provides the Process ID for every entry. In forensics, it is the primary method for identifying Established Sessions with foreign IPs. By matching the PID to a program in Task Manager, an investigator can determine exactly which software is responsible for potentially unauthorized data transmission.



- **Command netstat -an**

  The netstat -an command displays a complete list of all active TCP and UDP connections along with the ports the computer is listening on. The -a flag ensures all connections are shown, while the -n flag displays them numerically to avoid the delay of resolving hostnames. In forensics, it provides a "snapshot" of the network state, allowing an investigator to quickly spot unauthorized foreign IP addresses. It is the most basic yet essential command for verifying which ports are open to the network at any given moment.



2. Investigation

- **Listening Ports**

  On the Kali Linux host, netstat -tulpn revealed active listening services on **Port 23 (Telnet/inetd)**, **Port 80 (Apache2)**, and several **Docker-proxy** ports (9200, 1514, 1515). On the Windows host, ports **135, 445, and 139** were found in a LISTENING state, representing standard RPC and SMB services.

- **Established Sessions**

  Using netstat -ano on the Windows host, multiple **ESTABLISHED** sessions were identified. Specifically, a session was noted between the local interface (10.0.2.8) and the Kali VM (10.0.2.3) over the ports used during the earlier FTP/Telnet exercises. This confirms an active data exchange between the laptop and the target server

- **Suspicious Foreign Ips**

  The IP address **10.0.2.3** (Kali Linux) is flagged as a suspicious foreign IP in this context, as it is receiving plaintext credentials via Telnet and FTP. Additionally, several external IPs (e.g., 150.171.28.11) were observed connecting via Port 443; while these appear to be standard OS telemetry, they should be cross-referenced with Threat Intelligence feeds."

- **Which Process Owns the Connection**

  On Kali, the Telnet service is owned by **PID 760 (inetd)** and the Web service by **PID 758 (apache2)**. On the Windows host, the primary process responsible for the established network sessions was identified as **PID 8028**. It was of Telnet

.

- **Default Gateway**

  The routing table analyzed via netstat -rn on the Kali host identifies the default gateway as **10.0.2.1** on the eth0 interface. This gateway handles all traffic destined for the 0.0.0.0 network, serving as the exit point for the NAT Network environment

3. **Recommend Containment Steps**

- These are the tactical actions to stop the threat immediately.
- Immediate Service Termination: "Disable the Telnet service (inetd) on the Kali VM immediately using sudo systemctl stop openbsd-inetd to close Port 23. On the Windows host, use taskkill /PID 8028 /F to force-close the unauthorized connection."
- Host-Based Firewalls: "Implement a rule in the Windows Firewall to block all inbound and outbound traffic to the suspicious IP 10.0.2.3 until the intent of the communication is fully verified."
- Protocol Hardening: "Replace clear-text protocols (Telnet/FTP) with secure alternatives such as SSH (Port 22) and SFTP. This ensures that even if traffic is intercepted, credentials cannot be harvested in plaintext."
- Network Isolation: "If the suspicious communication continues from other PIDs, isolate the host by placing it in a Quarantine VLAN or disconnecting the virtual network interface to prevent lateral movement."